

# SPI Driver Document

## 1. Driver Overview

This SPI driver provides a hardware abstraction layer for the SPI peripheral on the STM32F401 microcontroller. It encapsulates low-level register operations and exposes easy-to-use APIs for configuring SPI communication, transmitting and receiving data, and handling interrupts.

The driver supports:

- Master and slave modes
- Full-duplex communication
- Half-duplex communication
- Simplex receive-only communication
- Blocking data transfer
- Interrupt-driven data transfer
- Software and hardware NSS management
- SPI event notification through callback functions

The application interacts exclusively through the driver APIs and does not require direct access to SPI registers.

## 2. Level of Abstraction

The driver abstracts several hardware-specific operations:

| Hardware Feature                 | Driver Abstraction                    |
|----------------------------------|---------------------------------------|
| RCC clock enable registers       | <code>SPI_PeriClockControl()</code>   |
| SPI control registers (CR1, CR2) | <code>SPI_Init()</code>               |
| SPI enable/disable mechanism     | <code>SPI_PeripheralControl()</code>  |
| Status register flags            | <code>SPI_GetFlagStatus()</code>      |
| NVIC interrupt configuration     | <code>SPI_IRQInterruptConfig()</code> |
| NVIC priority configuration      | <code>SPI_IRQPriorityConfig()</code>  |

|                       |                                     |
|-----------------------|-------------------------------------|
| Data register access  | SPI_SendData()<br>SPI_ReceiveData() |
| Interrupt servicing   | SPI_IRQHandling()                   |
| Overrun flag clearing | SPI_ClearOVRFlag()                  |

This abstraction allows applications to use SPI without directly manipulating registers.

### 3. Driver Architecture

The driver is split into clear layers, each handling one part of the work. Lower layers deal with registers and hardware setup, while upper layers provide simple functions for the application. This makes the code easier to understand, reuse, and modify without touching low-level details.

The driver is organized into four major layers:

#### **Peripheral Configuration**

Responsible for:

- Enabling and disabling the SPI peripheral clock
- Initializing the SPI peripheral with user-specified parameters
- Resetting the SPI peripheral to its default state

Functions:

- SPI\_PeriClockControl()
- SPI\_Init()
- SPI\_DeInit()

#### **Peripheral Control**

Responsible for runtime control of the SPI peripheral.

Functions:

- SPI\_PeripheralControl()
- SPI\_SSConfig()
- SPI\_SSOEConfig()

Provides:

- SPI enable/disable
- Software NSS control
- Hardware NSS control

### **Data Transfer**

Supports polling-based communication.

Functions:

- SPI\_SendData()
- SPI\_ReceiveData()

Features:

- 8-bit frame format
- 16-bit frame format
- Blocking operation
- Automatic TXE and RXNE flag handling

### **Interrupt Management**

Supports non-blocking communication.

Functions:

- SPI\_SendDataIT()
- SPI\_ReceiveDataIT()
- SPI\_IRQHandling()

Interrupt sources handled:

- TXE (Transmit buffer empty)
- RXNE (Receive buffer not empty)
- OVR (Overrun error)

Helper ISR routines:

- spi\_txe\_interrupt\_handle()
- spi\_rxne\_interrupt\_handle()
- spi\_ovr\_err\_interrupt\_handle()

## 4. Important APIs

### **SPI\_Init()**

Configures:

- Device mode
- Bus configuration
- Clock speed
- Data frame format
- Clock polarity
- Clock phase
- Slave management

Used before enabling the SPI peripheral.

### **SPI\_SendData()**

Performs blocking transmission.

Operation:

1. Wait for TXE flag.
2. Write data into DR register.
3. Repeat until all bytes are sent.
4. Wait for BSY flag to clear.

### **SPI\_ReceiveData()**

Performs blocking reception.

Operation:

1. Wait for RXNE flag.
2. Read received data from DR.
3. Repeat until requested length is received.

### **SPI\_SendDataIT()**

Starts interrupt-based transmission.

Responsibilities:

- Stores transmit buffer information.
- Marks peripheral busy.
- Enables TXE interrupt.
- Enables error interrupt.

Actual transmission occurs inside ISR.

### **SPI\_ReceiveDataIT()**

Starts interrupt-based reception.

Responsibilities:

- Stores receive buffer information.
- Marks peripheral busy.
- Enables RXNE interrupt.
- Enables error interrupt.

Actual reception occurs inside ISR.

### **SPI\_IRQHandling()**

Central interrupt handler.

Handles:

- Transmission events
- Reception events
- Overrun errors

Calls application callback when transfer is completed.

## **5. Event Notification Mechanism**

The driver provides a weak callback function:

`SPI_ApplicationEventCallback()`

Applications can override this function to handle events such as:

`SPI_EVENT_TX_CMPLT`

`SPI_EVENT_RX_CMPLT`

`SPI_EVENT_OVR_ERR`

This creates a clean separation between driver and application logic.

## 6. Overrun Error Handling

The SPI peripheral generates an overrun error when new data arrives before previously received data is read.

The driver:

- Detects OVR flag in ISR.
- Clears the flag by reading DR followed by SR.
- Notifies the application through callback.

Functions involved:

- `spi_ovr_err_interrupt_handle()`
- `SPI_ClearOVRFlag()`

## 7. Conclusion

This SPI driver provides a complete abstraction for STM32 SPI communication. It supports both polling and interrupt-driven communication, handles peripheral configuration, interrupt management, and error detection internally, and exposes a clean API to the application layer. The callback mechanism further improves modularity by allowing event handling without modifying the driver source code.